

RESEARCH ARTICLE | FEBRUARY 12 2025

Runge–Kutta random feature method for solving multiphase flow problems of cells

Yangtao Deng (邓扬涛) ; Qiaolin He (贺巧琳)  



Physics of Fluids 37, 023350 (2025)

<https://doi.org/10.1063/5.0252273>



Articles You May Be Interested In

Composite group of explicit Runge-Kutta methods

AIP Conf. Proc. (June 2016)

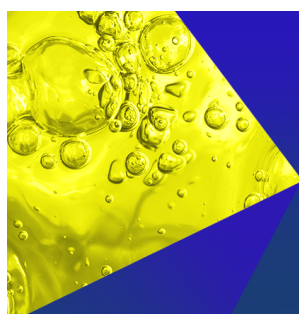
Construction of exponentially fitted symplectic Runge-Kutta-Nyström methods from partitioned Runge-Kutta methods

AIP Conf. Proc. (October 2014)

Efficiency of Runge-Kutta methods in solving Kepler problem

AIP Conf. Proc. (May 2017)

12 April 2025 15:13:14



Physics of Fluids
Special Topics
Open for Submissions

[Learn More](#)



Runge–Kutta random feature method for solving multiphase flow problems of cells

Cite as: Phys. Fluids **37**, 023350 (2025); doi: [10.1063/5.0252273](https://doi.org/10.1063/5.0252273)

Submitted: 8 December 2024 · Accepted: 16 January 2025 ·

Published Online: 12 February 2025



View Online



Export Citation



CrossMark

Yangtao Deng (邓扬涛) and Qiaolin He (贺巧琳)

AFFILIATIONS

School of Mathematics, Sichuan University, Chengdu, China

^{a)} Author to whom correspondence should be addressed: qlhejenny@scu.edu.cn

ABSTRACT

Cell collective migration plays a crucial role in a variety of physiological processes. In this work, we propose the Runge–Kutta random feature method to solve the nonlinear and strongly coupled multiphase flow problems of cells, in which the random feature method in space and the explicit Runge–Kutta method in time are utilized. Experiments indicate that this algorithm can effectively deal with time-dependent partial differential equations with strong nonlinearity and achieve high accuracy both in space and time. Moreover, in order to improve the computational efficiency and save computational resources, we choose to implement parallelization and non-automatic differentiation strategies in our simulations. We also provide error estimates for the Runge–Kutta random feature method, and a series of numerical experiments are shown to validate our method.

Published under an exclusive license by AIP Publishing. <https://doi.org/10.1063/5.0252273>

I. INTRODUCTION

Cell collective migration plays a vital role in essential physiological processes. Research work about how mechanical forces interact with collective cell behavior at the tissue level¹ has garnered growing interest across various fields. A phase field system (9) is used to model cells as active deformable droplets in two dimensions in Ref. 2. Although finite difference methods,³ finite volume methods,⁴ and finite element methods⁵ can be employed to solve this kind of partial differential equations (PDEs) and great success in theory and application have been achieved, these methods still face challenges such as complex geometric shapes, mesh generation, and lack of generalization. These approaches generally involve generating the grid of points or elements, discretizing differential and integral expressions and solving the resulting discrete equations. However, the complexity of the computational domain can make mesh generation very difficult,⁶ which showed that geometry complexity causes issues like distorted elements and long computation times, and the problem discretization could create a bias between the mathematical nature of the PDEs and its approximating model.⁷ Moreover, obtaining numerical values outside the computational domain requires using extrapolation methods or regenerating the grid and recalculating.³ The results of extrapolation are usually not accurate, and the costly process of grid regeneration may waste a lot of computational resources and time.

On the other hand, the success of deep learning in computer vision and natural language processing⁸ has attracted widespread

attention in the field of scientific computing. As a special class of functions, neural networks have been proven to be universal approximators of continuous functions.⁹ Many researchers are committed to using neural networks to solve ordinary differential equations (ODEs) and PDEs.^{10–16} Due to the different formulations of PDEs, deep Ritz methods (DRM),¹² deep Galerkin methods (DGM),¹³ physics-informed neural networks (PINN),¹⁵ and weak adversarial networks (WAN)¹⁴ have been proposed based on loss (objective) functions of variational, strong, and weak forms, respectively. Deep learning-based algorithms have become general methods to solve various PDEs without any type of mesh generation. However, there is a lack of reliable error control in existing deep learning methods. For example, in the absence of the exact solution, as the number of parameters increases, the numerical approximation provided by deep learning methods does not show a clear trend of convergence.¹⁷

Recent studies have shown that a specific type of neural network—extreme learning machine (ELM)¹⁸ or random features^{19,20}—can be used to solve ODEs and PDEs. ELM is a feedforward neural network with a single hidden layer, which randomly assigns weights and biases between the input layer and the hidden layer, and determines the weights of the output layer by analytical methods. In Ref. 21, the weights of the output layer are determined by minimizing the squared error between the network outputs and the target values using the least squares method. Due to the characteristics of this architecture, the weights and biases of the hidden layer do not need to be learned.

This makes ELM faster than typical deep neural networks, because optimization may lead to extremely slow learning speeds in deep neural networks. Overall, randomized neural networks improve the efficiency of learning tasks while maintaining high accuracy in numerical solutions. ELM is a mesh-free method, so it can easily handle the PDEs in complex geometric domains.^{22,23} In addition, ELM is a universal approximator,^{18,24} so it has the potential to represent any continuous function. In Ref. 25, ELM is developed to differential problems, where the authors implemented a model called physics-informed extreme learning machine (PIELM), and subsequently, ELM has been successfully applied to solve ODEs and PDEs.^{22,23,25–29}

In Ref. 27, a special type of partition of unity (PoU), called domain decomposition, is combined with the ELM to approximate the solution of PDEs, and a block time-marching strategy is proposed for long-time simulations, while a nonlinear least squares method is used to solve nonlinear differential equations. However, as the simulation time increases, the error will grow exponentially quickly,²³ in addition, when dealing with nonlinear differential equations that exhibit strong nonlinearity [e.g., (9)], the nonlinear least squares method is difficult to converge to satisfactory results. In Ref. 22, combining PoU and random feature functions, the random feature method (RFM) is proposed to solve static linear PDEs with complex geometric shapes. In Ref. 23, based on RFM, the space–time random feature method (ST-RFM) is proposed to solve linear time-dependent PDEs using random feature functions that depend on both spatial and temporal variables (STC), or the product of two random feature functions that depend on spatial and temporal variables separately (SoV). In addition, an automatic rescaling strategy is proposed to balance the weights of equations and boundary conditions. However, since the time variable is used as the input of the random feature functions, the ST-RFM requires a large amount of computational resources in the process and can only solve linear differential equations.

In this work, in order to solve the strongly nonlinear coupled multiphase flow problem of cells (9) (see Sec. II B) with less computational resources while maintaining high numerical accuracy, we choose to combine traditional numerical methods with deep learning-based methods. We use the RFM in space and the explicit Runge–Kutta (RK) method³⁰ in time, which is called the Runge–Kutta random feature method (RK-RFM). This new type of algorithm can achieve high accuracy in space and time. In addition, it is mesh-free, making them easy to use even in complex geometric settings. Moreover, this method can effectively solve PDEs with strong nonlinearity. At the same time, we choose to use the same set of random feature functions for different cells and manually derive the necessary neural network derivative functions before starting the calculations, instead of using automatic differentiation, which greatly accelerates computation and saves computational resources. We also provide the error estimates for the RK-RFM and show some numerical experiments.

The rest of this article is organized as follows. In Sec. II, we briefly introduce RFM and the multiphase flow model of cells. In Sec. III, we present the RK-RFM and give the error estimates of method. In Sec. IV, we show some numerical experiments to verify our theoretical results and the application of the RK-RFM in the multiphase flow problem of cells. The conclusions and remarks are given in Sec. V.

II. RANDOM FEATURE METHOD AND MULTIPHASE FLOW PROBLEM OF CELLS

A. Random feature method for static problems

Consider the following linear boundary-value problem:

$$\begin{cases} \mathcal{L}\phi(\mathbf{x}) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ \mathcal{B}\phi(\mathbf{x}) = g(\mathbf{x}), & \mathbf{x} \in \partial\Omega, \end{cases} \quad (1)$$

where $\mathcal{L}$ and $\mathcal{B}$ are linear differential and linear boundary operators, respectively. We use d and d_ϕ to denote the dimensions of $\mathbf{x}$ and $\phi = (\phi_1, \phi_2, \dots, \phi_{d_\phi})^T$, respectively.

In RFM,²² the domain Ω is divided into M_p non-overlapping sub-domains Ω_n with $\mathbf{x}_n$ as the centers. We have $\Omega = \bigcup_{n=1}^{M_p} \Omega_n$. For each Ω_n , the linear transformation is employed in RFM,

$$\tilde{\mathbf{x}} = \frac{1}{r_n}(\mathbf{x} - \mathbf{x}_n), \quad n = 1, \dots, M_p, \quad (2)$$

to map Ω_n into $[-1, 1]^d$, where $r_n \in R^d$ is the radius of Ω_n . A PoU function ψ_n is defined such that $\text{supp}(\psi_n) = \Omega_n$. Two commonly used PoU functions for $d = 1$ are

$$\psi(\mathbf{x}) = \mathbb{I}_{[-1,1]}(\tilde{\mathbf{x}}) \quad (3)$$

and

$$\begin{aligned} \psi(\mathbf{x}) = & \mathbb{I}_{[-\frac{5}{4}, -\frac{3}{4}]}(\tilde{\mathbf{x}}) \frac{1 + \sin(2\pi\tilde{\mathbf{x}})}{2} + \mathbb{I}_{[-\frac{3}{4}, \frac{1}{4}]}(\tilde{\mathbf{x}}) \\ & + \mathbb{I}_{[\frac{3}{4}, \frac{5}{4}]}(\tilde{\mathbf{x}}) \frac{1 - \sin(2\pi\tilde{\mathbf{x}})}{2}, \end{aligned} \quad (4)$$

where $\mathbb{I}_{[a,b]}(\mathbf{x}) = 1, \mathbf{x} \in [a, b]$ and $a \leq b$. When $d > 1$, the PoU function $\psi(\mathbf{x})$ is defined as $\psi(\mathbf{x}) = \prod_{i=1}^d \psi(x_i)$.

Next, random feature function ϕ_{nj} on Ω_n is formulated by a two-layer neural network,

$$\phi_{nj}(\mathbf{x}) = \sigma(\mathbf{W}_{nj} \cdot \tilde{\mathbf{x}} + b_{nj}), \quad j = 1, 2, \dots, J_n, \quad (5)$$

where σ is the nonlinear activation function, $\mathbf{W}_{nj}$ and b_{nj} are randomly chosen through uniform distribution $\mathbb{U}(-R_m, R_m)$ but fixed, R_m controls the magnitude of the parameters, and J_n is the number of random feature functions. In RFM, the approximate solution is formed by the linear combination of these random feature functions along with the PoU functions,

$$\tilde{\phi}(\mathbf{x}) = \left(\sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj}^1 \phi_{nj}^1(\mathbf{x}), \dots, \sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj}^{d_\phi} \phi_{nj}^{d_\phi}(\mathbf{x}) \right)^T, \quad (6)$$

where $u_{nj}^i \in \mathbb{R}$ is unknowns to be determined and $M = M_p J_n$ denotes the degree of freedom.

Then, RFM minimizes the loss function,

$$\begin{aligned} \text{Loss}(\{u_{nj}^i\}) = & \sum_{n=1}^{M_p} \left(\sum_{q=1}^Q \|\lambda_{n,q} (\mathcal{L}\tilde{\phi}(\mathbf{x}_q^n) - f(\mathbf{x}_q^n))\|_2^2 \right) \\ & + \sum_{n=1}^{M_p} \left(\sum_{\mathbf{x}_q^n \in \partial\Omega} \|\lambda_{n,b} (\mathcal{B}\tilde{\phi}(\mathbf{x}_q^n) - g(\mathbf{x}_q^n))\|_2^2 \right), \end{aligned} \quad (7)$$

though the linear least squares method. When employing the PoU function ψ defined in (3), it is necessary to incorporate regularization terms into the loss function (7) to ensure continuity between neighboring subdomains. Conversely, when utilizing ψ defined in (4), regularization terms are not necessary. To find the optimal set of parameters $\{u_{nj}^i\}$, RFM samples Q collocation points $\{\mathbf{x}_q^n\}$ in each Ω_n and computes the rescaling parameters $\lambda_{n,q} = \text{diag}(\lambda_{n,q}^1, \dots, \lambda_{n,q}^{d_\phi})$ and $\lambda_{n,b} = \text{diag}(\lambda_{n,b}^1, \dots, \lambda_{n,b}^{d_\phi})$. Specifically, the rescaling parameters are calculated using the following method:

$$\begin{aligned} \lambda_{n,q}^i &= \frac{c}{\max_{1 \leq j \leq J_n} |\mathcal{L}(\psi_n(\mathbf{x}_q^n) \varphi_{nj}^i(\mathbf{x}_q^n))|}, \\ q &= 1, \dots, Q, \quad n = 1, \dots, M_p, \quad i = 1, \dots, d_\phi, \\ \lambda_{n,b}^i &= \frac{c}{\max_{1 \leq j \leq J_n} |\mathcal{B}(\psi_n(\mathbf{x}_q^n) \varphi_{nj}^i(\mathbf{x}_q^n))|}, \\ \mathbf{x}_q^n &\in \partial\Omega, \quad n = 1, \dots, M_p, \quad i = 1, \dots, d_\phi, \end{aligned} \quad (8)$$

where $c > 0$ is a constant, see more details in Appendix. The numerical result is formulated by using Eq. (6).

B. Multiphase flow problem of cells

We consider the following time-dependent multiphase flow problem, which is a phase-field problem used to model cells as active deformable droplets as in Ref. 2,

$$\begin{cases} \partial_t \phi_i = -v_i \nabla \phi_i - \frac{\delta \mathcal{F}}{\delta \phi_i} = F(\mathbf{x}, t, \phi(\mathbf{x}, t)), \\ \mathbf{x}, t \in \Omega \times (0, T], \quad i = 1, \dots, d_\phi, \\ \mathcal{B}\phi(\mathbf{x}, t) = \mathbf{g}(\mathbf{x}, t), \quad \mathbf{x}, t \in \partial\Omega \times [0, T], \\ \phi(\mathbf{x}, 0) = \mathbf{h}(\mathbf{x}), \quad \mathbf{x} \in \Omega, \end{cases} \quad (9)$$

where $\mathcal{B}$ is a linear boundary operator and ϕ_i represents the i th cell, with $\phi_i = 1$ and $\phi_i = 0$ indicating the interior and exterior of the cell, respectively. The cell boundary is defined to locate at $\phi_i = \frac{1}{2}$, and $\mathcal{F}$ is a free energy with $\mathcal{F} = \mathcal{F}_{\text{CH}} + \mathcal{F}_{\text{area}} + \mathcal{F}_{\text{rep}}$, where

$$\begin{aligned} \mathcal{F}_{\text{CH}} &= \sum_i \frac{\gamma}{\lambda} \int \{4\phi_i^2(1 - \phi_i)^2 + \lambda^2(\nabla \phi_i)^2\} d\mathbf{x}, \\ \mathcal{F}_{\text{area}} &= \sum_i \mu \left(1 - \frac{1}{\pi R^2} \int \phi_i^2 d\mathbf{x}\right)^2, \\ \mathcal{F}_{\text{rep}} &= \sum_i \sum_{j \neq i} \frac{\kappa}{\lambda} \int \phi_i^2 \phi_j^2 d\mathbf{x}. \end{aligned} \quad (10)$$

The Cahn–Hilliard free energy, denoted as $\mathcal{F}_{\text{CH}}$, is instrumental in maintaining the stability of the cell interfaces. The term $\mathcal{F}_{\text{area}}$ acts as a soft regulation for the area occupied by each cell, aiming for an area close to πR^2 , with R representing the radius of the cell. This ensures that the cells have the capacity to be compressed. Additionally, the repulsion term, $\mathcal{F}_{\text{rep}}$, imposes a penalty on areas where two cells are in overlap. The elasticity parameter γ represents the stiffness of the cell membrane. A higher γ indicates a stronger resistance of the cell interface to deformation. It influences the shape and integrity of individual cells. For example, when γ is increased, cells are more likely to maintain their original shape as the energy cost for deforming the interface

becomes higher. The area constraint parameter μ determines how strongly the cells are restricted to maintain a certain area. A higher μ implies a stronger preference for the cells to have an area close to πR^2 . The interface width parameter λ defines the width of the cell interfaces at equilibrium. It is used to rescale the quantities in the model to ensure a certain degree of invariance under changes of the interface width. The equilibrium shape of the interfaces is related to λ , and it affects the overlap and interaction between cells. The repulsion parameter κ determines the strength of the repulsive force between cells. A higher κ indicates a stronger repulsion between cells when they approach each other.

Then, taking variation, we have

$$\begin{aligned} \frac{\delta \mathcal{F}_{\text{CH}}}{\delta \phi_i} &= \frac{8\gamma}{\lambda} \phi_i(1 - \phi_i)(1 - 2\phi_i) - 2\gamma\lambda \Delta \phi_i, \\ \frac{\delta \mathcal{F}_{\text{area}}}{\delta \phi_i} &= -\frac{4\mu}{\pi R^2} \phi_i \left(1 - \frac{1}{\pi R^2} \int \phi_i^2 d\mathbf{x}\right), \\ \frac{\delta \mathcal{F}_{\text{rep}}}{\delta \phi_i} &= \frac{2\kappa}{\lambda} \sum_{k \neq i} \phi_k^2 \phi_i. \end{aligned} \quad (11)$$

v_i is the total velocity of the i th cell, given by the following force balance equation:

$$\zeta v_i = F_i^{\text{int}}, \quad (12)$$

where ζ is a substrate friction coefficient and F_i^{int} is the total force acting on the interface of i th cell. These microscopic interface forces in terms of a macroscopic tissue stress tensor σ_{tissue} are defined as

$$F_i^{\text{int}} = \int \phi_i \nabla \cdot \sigma_{\text{tissue}} d\mathbf{x} = - \int \sigma_{\text{tissue}} \cdot \nabla \phi_i d\mathbf{x}, \quad (13)$$

where the tissue stress tensor σ_{tissue} is decomposed into passive and active stresses,

$$\sigma_{\text{tissue}} = -P\mathbb{I} - \zeta Q, \quad (14)$$

where the activity parameter ζ represents the strength of the internal driving force within cells. A positive ζ corresponds to extensile activity, while a negative one represents contractile activity, affecting the system's stability and behavior. The fields P and Q are the tissue pressure and tissue nematic tensor, defined as follows:

$$P = \sum_i \left(\frac{\delta \mathcal{F}_{\text{rep}}}{\delta \phi_i} - \frac{\delta \mathcal{F}_{\text{CH}}}{\delta \phi_i} - \frac{\delta \mathcal{F}_{\text{area}}}{\delta \phi_i} \right) \quad (15)$$

and

$$Q = \sum_i \phi_i S_i, \quad (16)$$

where S_i is the deformation tensor of i th cell defined as the traceless part of $-\int (\nabla \phi_i)^T \nabla \phi_i d\mathbf{x}$. Specifically, $S_i \equiv S(\phi_i)$, with

$$\begin{aligned} S(\phi) &= \begin{pmatrix} S_{11} & S_{12} \\ S_{12} & -S_{11} \end{pmatrix} \\ &= \int \begin{pmatrix} \frac{1}{2}((\partial_y \phi)^2 - (\partial_x \phi)^2) & -(\partial_x \phi)(\partial_y \phi) \\ -(\partial_x \phi)(\partial_y \phi) & \frac{1}{2}((\partial_x \phi)^2 - (\partial_y \phi)^2) \end{pmatrix} d\mathbf{x}. \end{aligned} \quad (17)$$

III. RUNGE-KUTTA RANDOM FEATURE METHOD AND ERROR ESTIMATION

A. Runge-Kutta random feature method

We now begin to introduce the RK-RFM. Given a partition of $[0, T] : 0 = t_0 < t_1 < \dots < t_K = T$ and for $k = 0, \dots, K-1$, the explicit scheme³⁰ is written as

$$\begin{cases} D_1(\mathbf{x}) = F(\mathbf{x}, t_k, \phi_{t_k}(\mathbf{x})), \\ D_2(\mathbf{x}) = F(\mathbf{x}, t_k + c_2 \Delta t, \phi_{t_k}(\mathbf{x}) + a_{21} \Delta t D_1(\mathbf{x})), \\ \vdots \\ D_p(\mathbf{x}) = F(\mathbf{x}, t_k + c_p \Delta t, \phi_{t_k}(\mathbf{x}) + \Delta t \sum_{i=1}^{p-1} a_{pi} D_i(\mathbf{x})), \\ \phi_{t_{k+1}}(\mathbf{x}) = \phi_{t_k}(\mathbf{x}) + \Delta t \sum_{i=1}^p b_i D_i(\mathbf{x}) \\ \equiv \phi_{t_k}(\mathbf{x}) + \Delta t H(\mathbf{x}, t_k, \phi_{t_k}(\mathbf{x}), \Delta t), \end{cases} \quad (18)$$

where a_{pi} , b_i , and c_i are coefficients corresponding to the explicit p th-order RK method.

We use RFM to represent the approximate solution $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ of $\phi_{t_{k+1}}(\mathbf{x})$,

$$\begin{aligned} \tilde{\phi}_{t_{k+1}}(\mathbf{x}) = & \left(\sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj,t_{k+1}}^1 \phi_{nj,t_{k+1}}^1(\mathbf{x}), \dots, \right. \\ & \left. \times \sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj,t_{k+1}}^{d_\phi} \phi_{nj,t_{k+1}}^{d_\phi}(\mathbf{x}) \right)^T. \end{aligned} \quad (19)$$

Similarly, we sample Q collocation points $\{\mathbf{x}_q^n\}$ in each Ω_n and compute the rescaling parameters $\lambda_{n,q,t_{k+1}} = \text{diag}(\lambda_{n,q,t_{k+1}}^1, \dots, \lambda_{n,q,t_{k+1}}^{d_\phi})$ and $\lambda_{n,b,t_{k+1}} = \text{diag}(\lambda_{n,b,t_{k+1}}^1, \dots, \lambda_{n,b,t_{k+1}}^{d_\phi})$ as in (8). Then, the RK-RFM minimizes the loss,

$$\begin{aligned} \text{Loss}(\{u_{nj,t_{k+1}}^i\}) &= \sum_{n=1}^{M_p} \left(\sum_{q=1}^Q \|\lambda_{n,q,t_{k+1}} (\tilde{\phi}_{t_{k+1}}(\mathbf{x}_q^n) - \phi_{t_{k+1}}(\mathbf{x}_q^n))\|_2^2 \right) \\ &+ \sum_{n=1}^{M_p} \left(\sum_{\mathbf{x}_q^n \in \partial\Omega} \|\lambda_{n,b,t_{k+1}} (\mathcal{B} \tilde{\phi}_{t_{k+1}}(\mathbf{x}_q^n) - \mathbf{g}(\mathbf{x}_q^n, t_{k+1}))\|_2^2 \right), \end{aligned} \quad (20)$$

to find the optimal coefficients $\mathbf{U}_{t_{k+1}} = (u_{nj,t_{k+1}}^i)^\top$ of $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ defined in (19). In this article, we choose to use the PoU function ψ defined in (3). Therefore, regularization terms should be added to the loss function (20) to ensure continuity between subdomains. Specifically, following the approach in Ref. 27, we impose C^1 continuity conditions at the interfaces between subdomains. The loss functional (20) implies

$$\mathcal{A}_{t_{k+1}} \mathbf{U}_{t_{k+1}} = \mathbf{f}_{t_{k+1}}, \quad (21)$$

where $\mathcal{A}_{t_{k+1}}$ is the coefficient matrix related to $\tilde{\phi}_{t_{k+1}}(\mathbf{x}_q^n)$ and $\mathbf{f}_{t_{k+1}}$ is the right-hand side term related to $\phi_{t_{k+1}}(\mathbf{x}_q^n)$ and $\mathbf{g}(\mathbf{x}_q^n, t_{k+1})$ (see the Appendix). Thus, we can obtain the optimal $\mathbf{U}_{t_{k+1}}$ by solving (21) using the linear least squares method. Once the optimal $\mathbf{U}_{t_{k+1}}$ is obtained, the $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ is used as the initial condition for the next time step in the RK method. We summarize the steps of the RK-RFM in Algorithm 1 and depict the architecture in Fig. 1.

Remark 1. (Without automatic differentiation) It should be noted that, with the PoU function ψ defined in (3), the calculation of the derivatives

$$\begin{aligned} \nabla_{\mathbf{x}}^l \tilde{\phi}_{t_{k+1}}(\mathbf{x}) = & \left(\sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj,t_{k+1}}^1 \nabla_{\mathbf{x}}^l \phi_{nj,t_{k+1}}^1(\mathbf{x}), \dots, \right. \\ & \left. \times \sum_{n=1}^{M_p} \psi_n(\mathbf{x}) \sum_{j=1}^{J_n} u_{nj,t_{k+1}}^{d_\phi} \nabla_{\mathbf{x}}^l \phi_{nj,t_{k+1}}^{d_\phi}(\mathbf{x}) \right)^T, \quad l = 1, \dots, mp, \end{aligned} \quad (22)$$

is essential when dealing with the m th-order equation in the explicit p th-order RK method. When the number of subdomains M_p , the

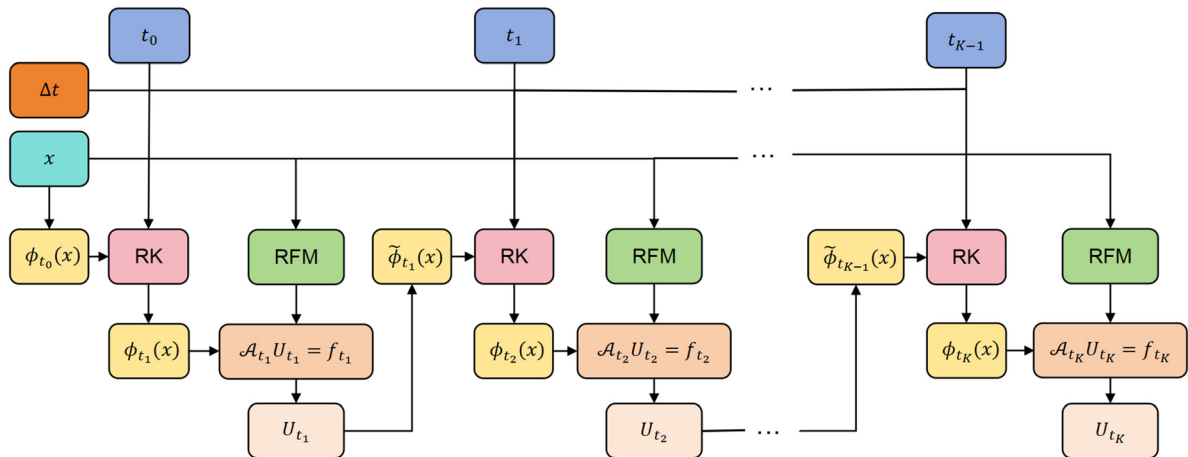


FIG. 1. The architecture of Algorithm 1.

ALGORITHM 1: The Runge–Kutta random feature method.

Input: number of the subdomains M_p ; number of the random feature functions on each subdomain J_n ; number of collocation points in each subdomain Q ; terminal time T ; number of time intervals K ; coefficients in the explicit p th-order RK method a_{pi} , b_i and c_i ; range of uniform distribution R_m ; constant coefficient of the rescaling parameters c .

Output: The optimal $U_{t_1}, \dots, U_{t_K}$;

- 1: Divide Ω into M_p non-overlapping subdomains Ω_n ;
- 2: Sample Q collocation points $\{\mathbf{x}_q^n\}$ in each Ω_n ;
- 3: Compute the derivative functions (23);
- 4: **for** $k = 0, 1, 2, \dots, K - 1$ **do**
- 5: Construct J_n random feature functions $\varphi_{nj,t_{k+1}}$ on Ω_n by the uniform distribution $\mathbb{U}(-R_m, R_m)$;
- 6: Compute $\phi_{t_{k+1}}(\mathbf{x})$ according to (18);
- 7: Assemble the loss matrix (24) and solve it by the linear least squares method to obtain the optimal $U_{t_{k+1}}$;
- 8: Compute $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ according to (19);
- 9: $\phi_{t_{k+1}}(\mathbf{x}) := \tilde{\phi}_{t_{k+1}}(\mathbf{x})$
- 10: **end**
- 11: Return the optimal $U_{t_1}, \dots, U_{t_K}$.

number of random feature functions J_n , the number of sample points Q , and the order of the RK method p or the order of the equation m are large, using automatic differentiation can consume a large amount of computational resources and take a long time when we calculate (22). It is better to manually derive the necessary derivative functions,

$$\nabla_{\mathbf{x}}^l \varphi(\mathbf{x}, \mathbf{W}, b) = \nabla_{\mathbf{x}}^l \sigma(\mathbf{W} \cdot \tilde{\mathbf{x}} + b), \quad l = 1, \dots, mp, \quad (23)$$

before starting the computation and store them. This places high demands on the activation functions. In practice, we choose functions with the property of infinite differentiability as the activation functions. By this way, we can directly obtain $\nabla_{\mathbf{x}}^l \varphi_{nj,t_{k+1}}^i(\mathbf{x}) = \nabla_{\mathbf{x}}^l \varphi(\mathbf{x}, \mathbf{W}_{nj,t_{k+1}}^i, b_{nj,t_{k+1}}^i)$ by using (23) when needed, which will significantly reduce the use of computational resources and improve computation efficiency.

Remark 2. (Parallelization) In the aforementioned process, we need to generate different random feature functions $\varphi_{nj,t_{k+1}}^i$ for different $\phi_{t_{k+1}}^i$ and calculate the corresponding coefficient matrices and right-hand side terms. However, due to the limitations of computational resources, we need to solve for $U_{t_{k+1}}^i$ using the linear least squares method sequentially. When the dimension d_ϕ of $\phi_{t_{k+1}}^i$ is very large, this process becomes very time-consuming. To improve the computational efficiency and reduce the consumption of computational resources, we choose to use the same random feature functions $\varphi_{nj,t_{k+1}}$ for different $\phi_{t_{k+1}}^i$. This means that in each time step t_{k+1} , we only need to calculate the coefficient matrix $\mathcal{A}_{t_{k+1}}$ once, and the right-hand side terms $\mathbf{f}_{t_{k+1}}^i$ corresponding to different ϕ_i at time t_{k+1} are assembled into a matrix. Specifically, we have

$$\begin{aligned} \mathcal{A}_{t_{k+1}} U_{t_{k+1}} &= \mathcal{A}_{t_{k+1}} \begin{bmatrix} U_{t_{k+1}}^1, \dots, U_{t_{k+1}}^{d_\phi} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{f}_{t_{k+1}}^1, \dots, \mathbf{f}_{t_{k+1}}^{d_\phi} \end{bmatrix} \\ &= \mathbf{f}_{t_{k+1}}. \end{aligned} \quad (24)$$

Then, we only need to use the linear least squares method once to obtain the optimal coefficients $U_{t_{k+1}}$ corresponding to $\tilde{\phi}_{t_{k+1}}$.

B. Error estimates of Runge–Kutta random feature method

Lemma III. 1. Let σ be a bounded nonconstant piecewise continuous activation function, $d_\phi = 1$, $G_M(\mathbf{x})$ be of the form (6), and $M = M_p J_n$ denote the degree of freedom. Then, for any $f \in C(\Omega)$, there exists a sequence of $G_M(\mathbf{x})$, such that

$$\lim_{M \rightarrow \infty} \|G_M(\mathbf{x}) - f(\mathbf{x})\|_{L^2} = 0, \quad \forall \mathbf{x} \in \Omega.$$

Proof. It is a direct consequence of Theorem 2.1 in Ref. 24. ■

We use δu to denote the difference between the numerical solution $\hat{U} = (\hat{u}_j)^\top$ and the optimal solution $\tilde{U} = (\tilde{u}_j)^\top$ in the subdomain Ω_n . Let $\{\mathbf{b}_j\}_{j=1}^{J_n}$ be a unitary orthogonal basis in $\mathbb{R}^{J_n}$, there exists $\{\delta u_j\}_{j=1}^{J_n}$ such that $\delta u = \sum_{j=1}^{J_n} \delta u_j \mathbf{b}_j$, where $\delta u_j, j = 1, \dots, J_n$, are independent and identically distributed random variables with

$$\mathbb{E}[\delta u_j] = 0, \quad \mathbb{E}[\delta u_j] = \mu > 0, \quad \mathbb{E}[(\delta u_j)^2] = \delta^2 > 0. \quad (25)$$

Lemma III. 2. For the RFM, we denote $\hat{U} = (\hat{u}_{nj}^i)^\top$ and $\tilde{U} = (\tilde{u}_{nj}^i)^\top$ as the numerical solution and the optimal solution obtained by minimizing the loss function (7), respectively. We have

$$\mathbb{E}_{\delta u} [\|\hat{U} - \tilde{U}\|_{L^2}] \leq \sqrt{d_\phi M_p J_n} \delta,$$

where δ is defined in (25).

Proof.

$$\begin{aligned} (\mathbb{E}_{\delta u} [\|\hat{U} - \tilde{U}\|_{L^2}])^2 &\leq \mathbb{E}_{\delta u} [\|\hat{U} - \tilde{U}\|_{L^2}^2] \\ &= \mathbb{E}_{\delta u} \left[\sum_{i=1}^{d_\phi} \sum_{n=1}^{M_p} \left\| \sum_{j=1}^{J_n} \delta u_{nj}^i \mathbf{b}_j \right\|_{L^2}^2 \right] \\ &= d_\phi M_p J_n \delta^2. \end{aligned}$$

Lemma III. 3. Let two functions $\hat{G}_M(\mathbf{x})$ and $\tilde{G}_M(\mathbf{x})$ be of the formulation (6) and the coefficients being $\hat{U} = (\hat{u}_{nj}^i)^\top$, $\tilde{U} = (\tilde{u}_{nj}^i)^\top$, respectively, then there exists $C_1 \geq C_2 > 0$ such that

$$C_1 \|\hat{U} - \tilde{U}\|_{L^2} \geq \|\hat{G}_M(\mathbf{x}) - \tilde{G}_M(\mathbf{x})\|_{L^2} \geq C_2 \|\hat{U} - \tilde{U}\|_{L^2}.$$

Proof. Notice that

$$\begin{aligned} \|\hat{G}_M(\mathbf{x}) - \tilde{G}_M(\mathbf{x})\|_{L^2} &= \left(\sum_{i=1}^{d_\phi} \sum_{n=1}^{M_p} \sum_{j=1}^{J_n} (\hat{u}_{nj}^i - \tilde{u}_{nj}^i)^2 \int_{\Omega} \varphi_{nj}^i(\mathbf{x})^2 d\mathbf{x} \right)^{\frac{1}{2}} \\ &\leq C_1 \|\hat{U} - \tilde{U}\|_{L^2}, \end{aligned}$$

where $C_1 = \max_{n,j,i} \|\varphi_{nj}^i(\mathbf{x})\|_{L^2}$, and

$$\begin{aligned} \|\hat{G}_M(\mathbf{x}) - \tilde{G}_M(\mathbf{x})\|_{L^2} &= \left(\sum_{i=1}^{d_\phi} \sum_{n=1}^{M_p} \sum_{j=1}^{J_n} (\hat{u}_{nj}^i - \tilde{u}_{nj}^i)^2 \int_{\Omega} \varphi_{nj}^i(\mathbf{x})^2 d\mathbf{x} \right)^{\frac{1}{2}} \\ &\geq C_2 \|\hat{U} - \tilde{U}\|_{L^2}, \end{aligned}$$

where $C_2 = \min_{n,j,i} \|\varphi_{nj}^i(\mathbf{x})\|_{L^2}$. Moreover, for all $\varphi_{nj}^i \in \mathcal{L}^2(\Omega)$, we have $C_1, C_2 < +\infty$. ■

Theorem III. 1. Let $\phi_{t_k}^e$ be the exact solution of the equation and ϕ_{t_k} be the solution to explicit p th-order RK method (18). Let σ be a bounded nonconstant piecewise continuous activation function, ϕ_{t_k} and $\tilde{\phi}_{t_k}$ in the form of (19) are the optimal solution and the numerical

solution obtained by minimizing the least squares problem (24), respectively. We assume there exists $L > 0$ such that

$$\begin{aligned} & \|H(\mathbf{x}, t_k, \hat{\phi}_{t_k}(\mathbf{x}), \Delta t) - H(\mathbf{x}, t_k, \phi_{t_k}^e(\mathbf{x}), \Delta t)\|_{L^2} \\ & \leq L \|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}, \\ & \forall \hat{\phi}_{t_k}(\mathbf{x}), \quad \phi_{t_k}^e(\mathbf{x}) \in R^{d_\phi}, \quad \mathbf{x} \in \Omega, k = 0, \dots, K, \end{aligned}$$

where H is defined in (18) and given $\varepsilon > 0$, we also assume that

$$\|\tilde{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}(\mathbf{x})\|_{L^2} \leq \sqrt{d_\phi} \varepsilon, \quad \forall \mathbf{x} \in \Omega, k = 0, \dots, K, n = 1, \dots, M_p.$$

Then, there exists $C_1, C_2 > 0$ such that

$$\begin{aligned} & \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}] \\ & \leq \frac{e^{L(t_k - t_0)} - 1}{\Delta t L} \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon \right) + C_2 \Delta t^p \\ & = e_1(\Delta t) + e_2(\Delta t), \quad \forall \mathbf{x} \in \Omega, k = 0, \dots, K, \end{aligned} \quad (26)$$

where $e_1(\Delta t) = \frac{e^{L(t_k - t_0)} - 1}{\Delta t L} \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon \right)$ and $e_2(\Delta t) = C_2 \Delta t^p$. Proof: Notice that

$$\begin{aligned} & \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}] \\ & \leq \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_k}(\mathbf{x}) - \tilde{\phi}_{t_k}(\mathbf{x})\|_{L^2}] + \|\tilde{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}(\mathbf{x})\|_{L^2} \\ & \quad + \mathbb{E}_{\delta u} [\|\phi_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}]. \end{aligned}$$

Therefore, we have

$$\begin{aligned} & \|\phi_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2} \\ & = \|\phi_{t_k}(\mathbf{x}) - (\phi_{t_{k-1}}^e(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)) \\ & \quad + (\phi_{t_{k-1}}^e(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)) - \phi_{t_k}^e(\mathbf{x})\|_{L^2} \\ & = \|(\hat{\phi}_{t_{k-1}}(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \hat{\phi}_{t_{k-1}}(\mathbf{x}), \Delta t)) \\ & \quad - (\phi_{t_{k-1}}^e(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)) \\ & \quad + (\phi_{t_{k-1}}^e(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)) - \phi_{t_k}^e(\mathbf{x})\|_{L^2} \\ & \leq \|\hat{\phi}_{t_{k-1}}(\mathbf{x}) - \phi_{t_{k-1}}^e(\mathbf{x})\|_{L^2} \\ & \quad + \Delta t \|H(\mathbf{x}, t_{k-1}, \hat{\phi}_{t_{k-1}}(\mathbf{x}), \Delta t) - H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)\|_{L^2} \\ & \quad + \|(\phi_{t_{k-1}}^e(\mathbf{x}) + \Delta t H(\mathbf{x}, t_{k-1}, \phi_{t_{k-1}}^e(\mathbf{x}), \Delta t)) - \phi_{t_k}^e(\mathbf{x})\|_{L^2} \\ & \leq (1 + \Delta t L) \|\hat{\phi}_{t_{k-1}}(\mathbf{x}) - \phi_{t_{k-1}}^e(\mathbf{x})\|_{L^2} + C \Delta t^{p+1}. \end{aligned}$$

Then,

$$\begin{aligned} & \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}] \\ & \leq C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon + C_2 \Delta t^{p+1} \\ & \quad + (1 + \Delta t L) \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_{k-1}}(\mathbf{x}) - \phi_{t_{k-1}}^e(\mathbf{x})\|_{L^2}] \\ & = (1 + (1 + \Delta t L) + \dots + (1 + \Delta t L)^{k-1}) \\ & \quad \times \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon + C_2 \Delta t^{p+1} \right) \\ & \quad + (1 + \Delta t L)^k \mathbb{E}_{\delta u} [\|\hat{\phi}_{t_0}(\mathbf{x}) - \phi_{t_0}^e(\mathbf{x})\|_{L^2}] \\ & = \frac{(1 + \Delta t L)^k - 1}{\Delta t L} \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon + C_2 \Delta t^{p+1} \right) \\ & \leq \frac{(e^{\Delta t L})^k - 1}{\Delta t L} \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon + C_2 \Delta t^{p+1} \right) \\ & = \frac{e^{L(t_k - t_0)} - 1}{\Delta t L} \left(C_1 \sqrt{d_\phi M_p J_n} \delta + \sqrt{d_\phi} \varepsilon \right) + C_2 \Delta t^p. \end{aligned}$$

Remark 3. (The choice of Δt) Note that the error (26) consists of two parts, where $e_1(\Delta t)$ represents the error caused by RFM and optimization error, which is inversely proportional to Δt , and $e_2(\Delta t)$ represents the error introduced by the explicit p th-order RK method (18), which is directly proportional to Δt^p . A larger Δt can lead to a larger $e_2(\Delta t)$ and smaller $e_1(\Delta t)$, while a smaller Δt may result in a smaller $e_2(\Delta t)$ and larger $e_1(\Delta t)$, also costs much computational time. Ideally, Δt should be selected to satisfy $e_1(\Delta t) \approx e_2(\Delta t)$. However, this principle may not always be feasible since the values of δ and ε are often unknown. Regardless, this principle still provides valuable guidance for the selection of the time step Δt .

IV. NUMERICAL EXPERIMENTS

In this section, we present some numerical experiments to validate our methods. For quantitative comparison, we compare the errors of the numerical solution $\hat{\phi}_{t_k}(\mathbf{x})$ and the exact solution $\phi_{t_k}^e(\mathbf{x})$ in the relative L^∞ norm and relative L^2 norm, which are defined as

$$\begin{aligned} \|e\|_{L^\infty} &= \frac{\|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^\infty}}{\|\phi_{t_k}^e(\mathbf{x})\|_{L^\infty}}, \\ \|e\|_{L^2} &= \frac{\|\hat{\phi}_{t_k}(\mathbf{x}) - \phi_{t_k}^e(\mathbf{x})\|_{L^2}}{\|\phi_{t_k}^e(\mathbf{x})\|_{L^2}}. \end{aligned}$$

In two dimension, we set $M_p = N_x N_y$ and $Q = Q_x Q_y$, where N_x and N_y represent the number of subdomains along the x and y directions, respectively, and Q_x and Q_y are the number of points in the x and y directions of the subdomain Ω_n . The arrangement of subdomains and the distribution of collocation points in subdomain Ω_n are shown in Fig. 2.

In the linear least squares simulations, we use the least squares solver `torch.linalg.lstsq` available in PyTorch. We set the constant coefficient of the rescaling parameters $c = 100$. The second-order explicit RK method ($p = 2$) is utilized. In our experiments, we use AMD EPYC 7T83 CPU and NVIDIA RTX 4090 GPU for simulation.

A. Convergence test

We first test the convergence of our method. The exact solution of the multiphase flow problem (9) is given by

$$\begin{cases} \phi_1(\mathbf{x}, t) = \sin(x_1) \sin(x_2) \exp(-t), \\ \phi_2(\mathbf{x}, t) = \cos(x_1) \cos(x_2) \exp(-t), \end{cases} \quad (27)$$

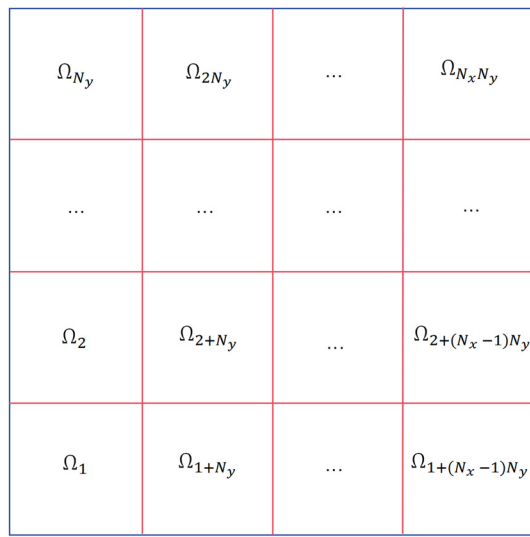
for $(\mathbf{x}, t) = (0, 2\pi)^2 \times (0, T]$ with initial condition,

$$\begin{cases} \phi_1(\mathbf{x}, 0) = \sin(x_1) \sin(x_2), \\ \phi_2(\mathbf{x}, 0) = \cos(x_1) \cos(x_2), \end{cases} \quad (28)$$

and Dirichlet boundary condition on $\partial\Omega$,

$$\begin{cases} \phi_1(\mathbf{x}, t) = \sin(x_1) \sin(x_2) \exp(-t), \\ \phi_2(\mathbf{x}, t) = \cos(x_1) \cos(x_2) \exp(-t). \end{cases} \quad (29)$$

We set $T = 1.0$, $R_m = 1.7$, $\gamma = \mu = \kappa = 0.001$, and $R = \lambda = \xi = \zeta = 1.0$. Algorithm 1 is employed to estimate $\phi(\mathbf{x}, t) = (\phi_1, \phi_2)^T$ with tan h and cos activation functions. The numerical results of the errors for ϕ with different Δt are shown in Table I and Fig. 3, where Q_{test} is the number of test points. From Table I and Fig. 3, for the tan h



(a) The arrangement of subdomains.

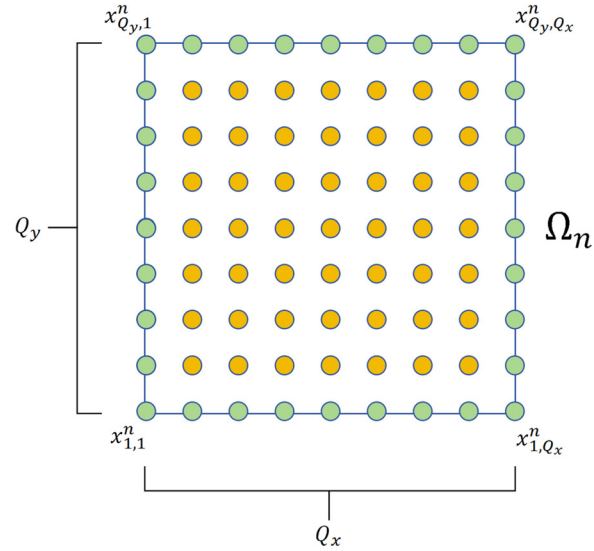

 (b) The distribution of collocation points in subdomain Ω_n .

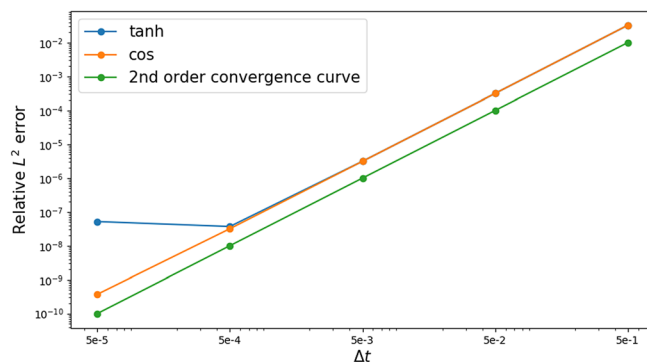
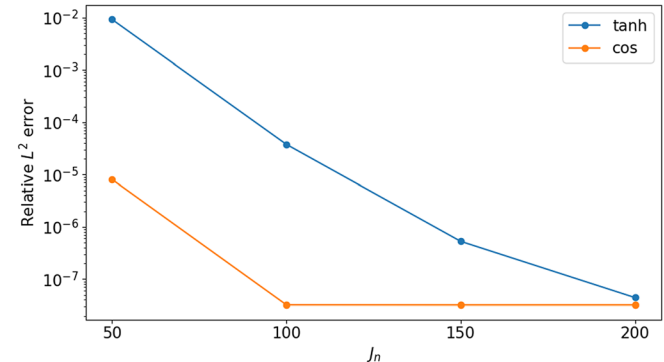
FIG. 2. The arrangement of subdomains and the distribution of collocation points in subdomain Ω_n : (a) the blue lines and red lines represent the boundaries and the interfaces between subdomains, respectively, and (b) the green points and yellow points represent the boundary points and interior points in each subdomain, respectively.

TABLE I. Relative L^∞ and L^2 errors for (27) obtained by Algorithm 1 with different Δt .

M_p	J_n	Q	Q_{test}	σ	Δt	5×10^{-1}	5×10^{-2}	5×10^{-3}	5×10^{-4}	5×10^{-5}
3×3	200	20×20	40×40	tanh	$\ e\ _{L^\infty}$	3.60×10^{-2}	3.53×10^{-4}	3.58×10^{-6}	1.45×10^{-7}	3.18×10^{-7}
					$\ e\ _{L^2}$	3.25×10^{-2}	3.15×10^{-4}	3.15×10^{-6}	3.70×10^{-8}	5.18×10^{-8}
					Time (s)	0.75	5.28	50.65	500.75	5023.06
				cos	$\ e\ _{L^\infty}$	4.43×10^{-2}	3.58×10^{-4}	3.58×10^{-6}	3.79×10^{-8}	5.70×10^{-10}
					$\ e\ _{L^2}$	3.26×10^{-2}	3.15×10^{-4}	3.14×10^{-6}	3.18×10^{-8}	3.71×10^{-10}
					Time (s)	0.81	4.98	47.46	470.18	4653.23

activation function, it can be observed that when $\Delta t = 5 \times 10^{-1}, 5 \times 10^{-2}$ and 5×10^{-3} , $e_1(\Delta t) < e_2(\Delta t)$, and the relative L^∞ and L^2 errors achieve second-order convergence. When $\Delta t = 5 \times 10^{-4}$, $e_1(\Delta t) \approx e_2(\Delta t)$, and the relative L^∞ and L^2 errors reach the

minimum in the five experiments. When $\Delta t = 5 \times 10^{-5}$, $e_1(\Delta t) > e_2(\Delta t)$, the relative L^∞ and L^2 errors begin to increase. For the cos activation function, it can be observed that when $\Delta t = 5 \times 10^{-1}, 5 \times 10^{-2}, 5 \times 10^{-3}$ and 5×10^{-4} , $e_1(\Delta t) < e_2(\Delta t)$, and the relative L^∞


FIG. 3. Relative L^2 error for (27) obtained by Algorithm 1 with $\Delta t = 5 \times 10^{-1}, 5 \times 10^{-2}, 5 \times 10^{-3}, 5 \times 10^{-4}$, and 5×10^{-5} .

FIG. 4. Relative L^2 error for (27) obtained by Algorithm 1 with $J_n = 50, 100, 150$, and 200.

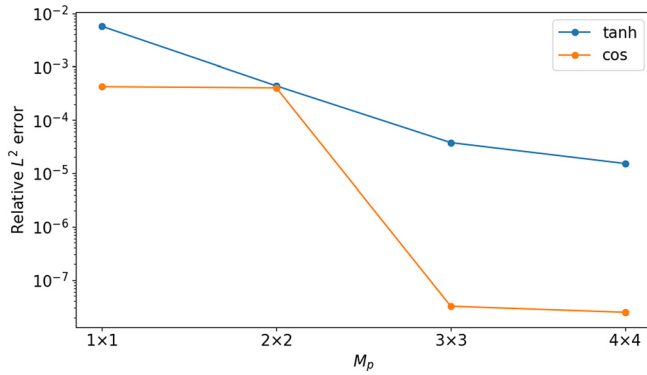


FIG. 5. Relative L^2 error for (27) obtained by Algorithm 1 with $M_p = 1 \times 1, 2 \times 2, 3 \times 3$, and 4×4 .

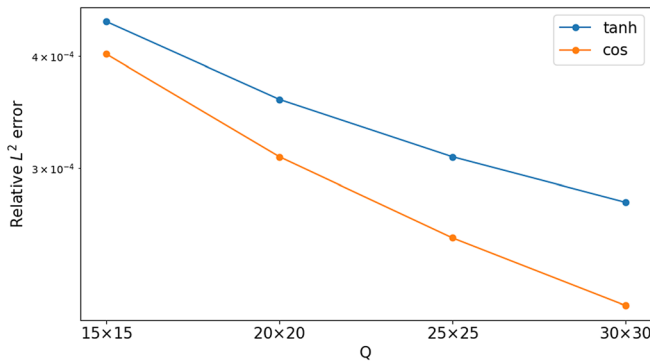


FIG. 6. Relative L^2 error for (27) obtained by Algorithm 1 with $Q = 15 \times 15, 20 \times 20, 25 \times 25$, and 30×30 .

TABLE II. Relative L^∞ and L^2 errors for (27) obtained by Algorithm 1 with different J_n .

M_p	Q	Q_{test}	Δt	σ	J_n	50	100	150	200
3×3	15×15	30×30	5×10^{-4}	tan h	$\ e\ _{L^\infty}$	1.75×10^{-2}	1.01×10^{-4}	2.88×10^{-6}	3.38×10^{-7}
					$\ e\ _{L^2}$	9.21×10^{-3}	3.76×10^{-5}	5.27×10^{-7}	4.40×10^{-8}
					Time (s)	224.44	271.97	301.70	370.18
				cos	$\ e\ _{L^\infty}$	1.92×10^{-5}	4.02×10^{-8}	3.93×10^{-8}	3.91×10^{-8}
					$\ e\ _{L^2}$	8.06×10^{-6}	3.21×10^{-8}	3.20×10^{-8}	3.20×10^{-8}
					Time (s)	202.81	278.36	319.03	344.35

TABLE III. Relative L^∞ and L^2 errors for (27) obtained by Algorithm 1 with different M_p .

J_n	Q	Q_{test}	Δt	σ	M_p	1×1	2×2	3×3	4×4
100	15×15	30×30	5×10^{-4}	tan h	$\ e\ _{L^\infty}$	1.72×10^{-2}	9.00×10^{-4}	1.01×10^{-4}	4.84×10^{-5}
					$\ e\ _{L^2}$	5.68×10^{-3}	4.37×10^{-4}	3.76×10^{-5}	1.52×10^{-5}
					Time (s)	100.12	151.22	271.97	551.76
				cos	$\ e\ _{L^\infty}$	7.61×10^{-4}	4.68×10^{-4}	4.02×10^{-8}	3.60×10^{-8}
					$\ e\ _{L^2}$	4.22×10^{-4}	4.02×10^{-4}	3.21×10^{-8}	2.47×10^{-8}
					Time (s)	95.07	140.55	278.36	513.32

and L^2 errors achieve second-order convergence. When $\Delta t = 5 \times 10^{-5}$, $e_1(\Delta t) \approx e_2(\Delta t)$, and the relative L^∞ and L^2 errors reach the minimum in the five experiments.

In order to verify the universal approximation using our method, we, respectively, test Algorithm 1 with different J_n , M_p , and Q when $\Delta t = 5 \times 10^{-4}$. The numerical results of the errors for ϕ are shown in Tables II–IV, Figs. 4–6, respectively. From Tables II and III, Figs. 4 and 5, it can be observed that the relative L^∞ and L^2 errors decrease as J_n (number of the random feature functions on each subdomain) and M_p (number of the subdomains) increase, which is consistent with the Lemma III.1. From Table IV and Fig. 6, it can be observed that the relative L^∞ and L^2 errors decrease as Q (number of collocation points in each subdomain) increases, which implies that increasing Q can decrease δ , which is defined in (25).

B. Multiphase flow problem of cells

Now, we use the RK-RFM to solve the multiphase flow problem of cells (9) for $(x, t) = (0, 200)^2 \times (0, T]$ with initial conditions,

$$\phi_i(\mathbf{x}, 0) = \frac{1 + \tanh\left(\frac{r - \|\mathbf{x} - \mathbf{C}_i\|_{L^2}}{\lambda}\right)}{2}, \quad i = 1, \dots, d_\phi, \quad (30)$$

and periodic boundary condition,

$$\begin{cases} \phi(0, x_2, t) = \phi(200, x_2, t), \\ \phi(x_1, 0, t) = \phi(x_1, 200, t), \end{cases} \quad (31)$$

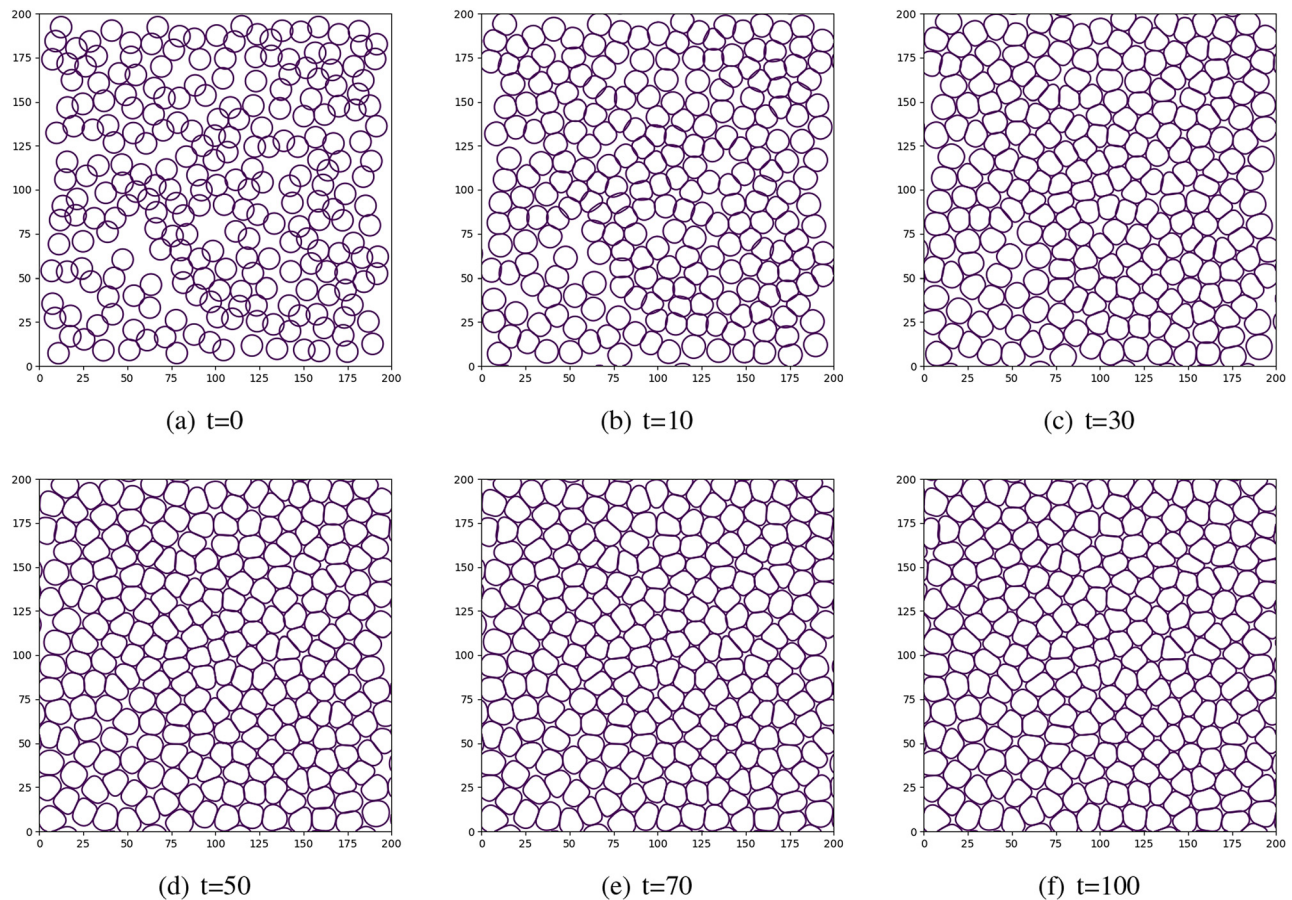
where r is the initial radius and $\mathbf{C}_i$ is the initial center of each cell. Algorithm 1 is employed to simulate the dynamic of $\phi(\mathbf{x}, t)$. We set $T = 100.0$, $R_m = 5.0$, $d_\phi = 240$, $r = 6.0$, $\gamma = 0.01$, $\mu = 3$, $\kappa = 0.1$, $\lambda = 2.5$, $R = 8.0$, $\mu = 3$, $\xi = 2$, and $\zeta = 0.005$. The parameters of Algorithm 1 are chosen to be $\Delta t = 0.1$, $M_p = 4 \times 4$, $Q = 40 \times 40$,

TABLE IV. Relative L^∞ and L^2 errors for (27) obtained by Algorithm 1 with different Q .

M_p	J_n	Δt	σ	Q	15×15	20×20	25×25	30×30
				Q_{test}	30×30	40×40	50×50	60×60
2×2	100	5×10^{-4}	tan h	$\ e\ _{L^\infty}$	9.00×10^{-4}	8.36×10^{-4}	7.81×10^{-4}	7.41×10^{-4}
				$\ e\ _{L^2}$	4.37×10^{-4}	3.58×10^{-4}	3.09×10^{-4}	2.75×10^{-4}
				Time (s)	151.22	173.75	198.86	259.54
			cos	$\ e\ _{L^\infty}$	4.68×10^{-4}	3.68×10^{-4}	3.01×10^{-4}	2.55×10^{-4}
				$\ e\ _{L^2}$	4.02×10^{-4}	3.09×10^{-4}	2.51×10^{-4}	2.11×10^{-4}
				Time (s)	140.55	161.65	188.70	250.59

$J_n = 600$, and $Q_{test} = 200 \times 200$. The tanh function is taken as the activation function. Initially, we randomly generate cells with radius r and center C_i in Ω . The area of the cells πr^2 is smaller than the target area of πR^2 . We allow them to partially overlap and relax during a period of inactivity. This simulation costs 1 h 37 min. The dynamical behavior of the cells is visually shown in Fig. 7, which shows the evolution of cells over time. Each cell is simulated as an active deformable droplet. As the simulation progresses, we can observe that the cells gradually increase in size under the effect of the term $\mathcal{F}_{area}$. When two

cells approach each other, the term $\mathcal{F}_{rep}$ generates a repulsive force to push them apart, and meanwhile, the term $\mathcal{F}_{CH}$ maintains the stability of the cell interfaces. The cells move in the direction of the total force F^{int} acting on the interfaces and gradually arrange themselves into a more ordered structure. Eventually, the system reaches a steady state. We also simulate the evolution of cells with different γ and κ , and other parameters and settings are the same as the example above. The steady state of the cells at $t = 100$ is shown in Fig. 8. It can be seen that the larger the γ , the stronger the resistance of the cell interfaces to

FIG. 7. The evolution of the cells at $t = 0, 10, 30, 50, 70, 100$.

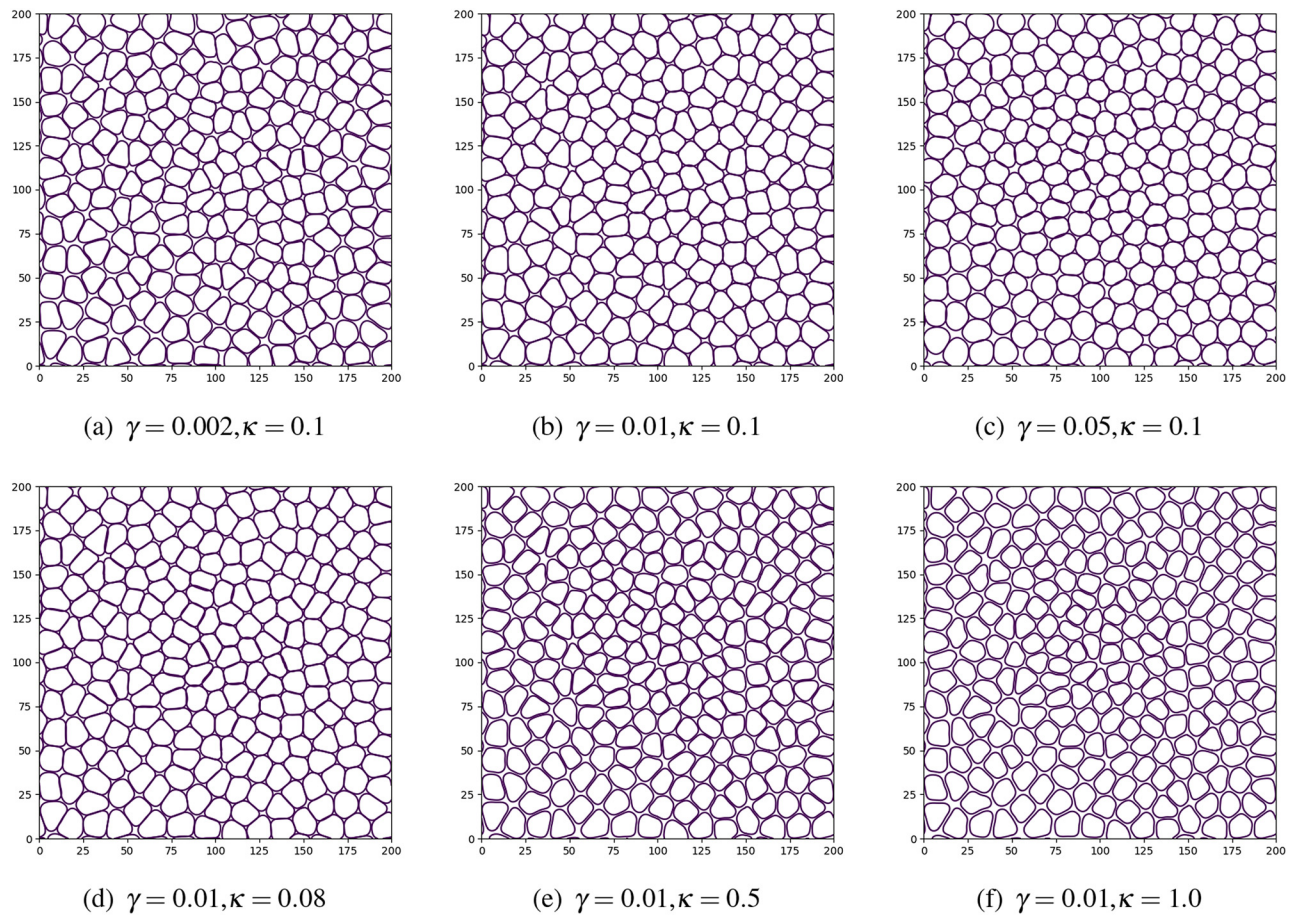


FIG. 8. The steady state of the cells with different γ and κ at $t = 100$: (a)–(c) results with fixed $\kappa = 0.1$ and (d)–(f) results with fixed $\gamma = 0.01$.

deformation, and the cells are more rounded. Additionally, the larger the κ , the greater the repulsive force between cells, and the larger the spacing between cells.

In order to further verify the cells evolution obtained by [Algorithm 1](#), we define the tissue velocity as $\mathbf{v} = \sum_{i=1}^{d_\phi} \phi_i \mathbf{v}_i$. Then, we measure the root mean square velocity $v_{\text{rms}} = (\langle v^2 \rangle)^{1/2}$ and nematic order $S_{\text{rms}} = (\langle \det S^2 \rangle)^{1/2} = (\langle S_{11}^2 + S_{12}^2 \rangle)^{1/2}$, the square roots of the sums of the squares of the means of the individual components of the velocity $(\langle v_x \rangle^2 + \langle v_y \rangle^2)^{1/2}$ and order $(\langle S_{11} \rangle^2 + \langle S_{12} \rangle^2)^{1/2}$, the mean angle of the director $\langle \omega \rangle = \langle \arctan(S_{12}/S_{11}) \rangle$, and the mean velocity $\langle \mathbf{v} \rangle$ separately with different γ and ζ . Specifically, we set $\Omega = (0, 100)^2$, $T = 1000.0$, $R_m = 5.0$, $d_\phi = 60$, $r = 6.0$, $\mu = 3$, $\kappa = 0.1$, $\lambda = 2.5$, $R = 8.0$, $\mu = 3$, and $\zeta = 2$. The parameters of [Algorithm 1](#) are chosen to be $\Delta t = 1.0$, $M_p = 3 \times 3$, $Q = 100 \times 100$, $J_n = 600$, and $Q_{\text{test}} = 200 \times 200$ with the initial condition (30) and periodic boundary conditions. The tanh function is taken as the activation function. The numerical results are shown in [Fig. 9](#). It is observed that the values v_{rms} and $S_{\text{rms}} - S_{\text{rms}}^0$ develop nonzero values with increasing activity ζ , where $S_{\text{rms}}^0 = S_{\text{rms}}|_{\zeta=0}$. The values $(\langle v_x \rangle^2 + \langle v_y \rangle^2)^{1/2}$ and $(\langle S_{11} \rangle^2 + \langle S_{12} \rangle^2)^{1/2}$ do not exhibit a significant rise with increasing ζ . The

mean angle $\langle \omega \rangle$ is uniformly distributed in $[-\pi, \pi]$ so that its average value is almost zero. The changing trends of the values shown in [Fig. 9](#) are consistent with the results in [Ref. 2](#).

Finally, we investigate the temporal evolution of v_{rms} and S_{rms} below or above the activity threshold. The activity threshold will emerge because cells must exert a strong enough push (or pull) to induce a sufficient deformation in their neighboring cells.² In addition, we investigate the temporal evolution of $(\langle v_x \rangle^2 + \langle v_y \rangle^2)^{1/2}$ and $(\langle S_{11} \rangle^2 + \langle S_{12} \rangle^2)^{1/2}$ with $\zeta < 0$. Parameters and settings are the same as the example above. The temporal changes in v_{rms} and S_{rms} below and above the activity threshold are shown in [Fig. 10](#). In both cases, the initial relaxation from the base setup is noticeable. It is observed that the system reverts to equilibrium when the activity level is below the threshold. Conversely, when the activity level is above the threshold, the system can maintain large flows and exhibits a continuous increase in its order. From [Fig. 11](#), it can be seen that the system is stable when $\zeta < 0$, which implies that the interactions at the cell interfaces tend to restore the stable state. The fluctuating patterns of the data depicted in [Fig. 11](#) align with the results in [Ref. 2](#). Since the system is initialized by creating many round cells defined in (30), which is different from the initial

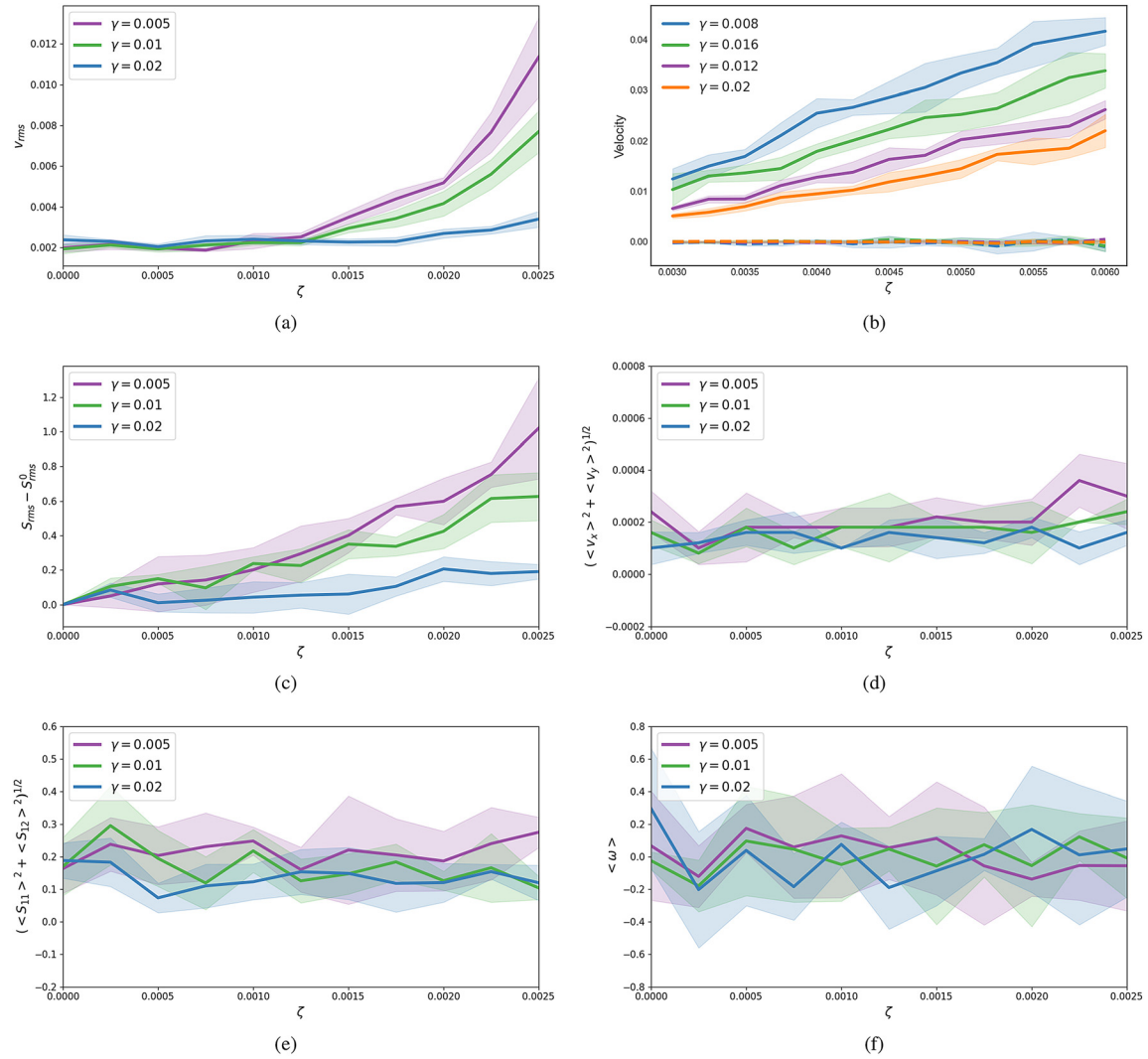


FIG. 9. The values of v_{rms} , $\langle v \rangle$, $S_{rms} - S_{rms}^0$, $(\langle v_x \rangle^2 + \langle v_y \rangle^2)^{1/2}$, $(\langle S_{11} \rangle^2 + \langle S_{12} \rangle^2)^{1/2}$, and $\langle \omega \rangle$ with different γ and ζ at $t = 1000.0$. (b) v_{rms} (plain lines) and $\langle v \rangle$ (dashed lines). Mean $\pm$ std from five simulations.

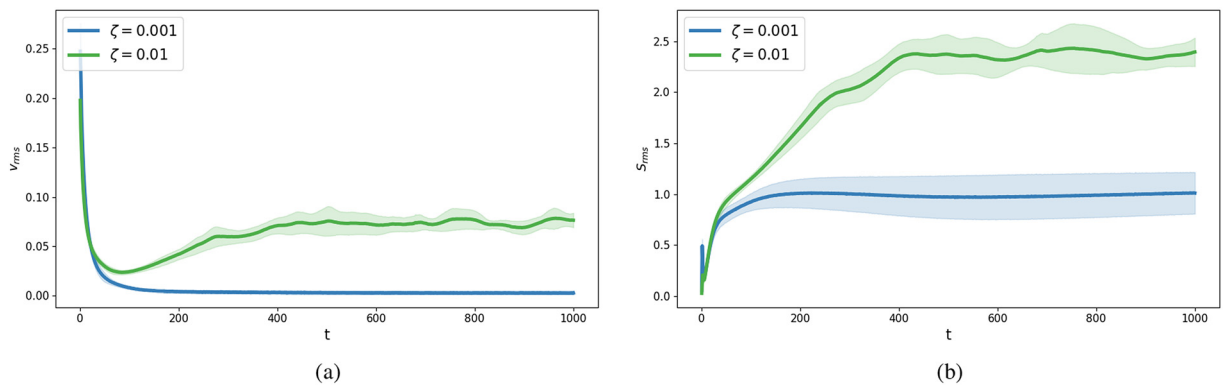


FIG. 10. The temporal evolution of v_{rms} and S_{rms} below (blue) and above (green) the activity threshold. Mean $\pm$ std from five simulations.

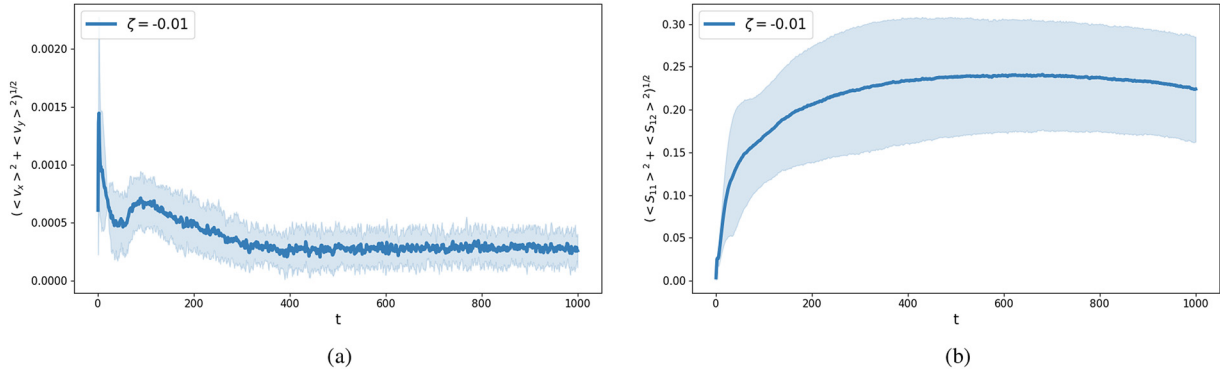


FIG. 11. The temporal evolution of $(\langle v_x \rangle^2 + \langle v_y \rangle^2)^{1/2}$ and $(\langle S_{11} \rangle^2 + \langle S_{12} \rangle^2)^{1/2}$ with $\zeta = -0.01$. Mean $\pm$ std from five simulations.

setting in Ref. 2, the value of nematic order increases for some initial time and attains its steady state.

V. CONCLUSIONS AND REMARKS

In this work, we present the Runge–Kutta random feature method (RK-RFM), a novel method designed to address the strongly nonlinear coupled multiphase flow problems of cells, while maintaining high numerical accuracy with reduced computational resources. The RK-RFM employs the RFM in space and the explicit RK method in time, achieving high accuracy in both space and time. This algorithm is mesh-free, making it suitable for complex geometric configurations, and is effective in solving PDEs with strong nonlinearity. Furthermore, we choose to use the same random feature functions for different cells and manually derive the necessary neural network derivative functions before the calculation begins, instead of relying on automatic differentiation, which significantly accelerates computation and conserves computational resources. We also provide error estimates for the RK-RFM and present some numerical experiments to validate our method and the application of RK-RFM in the multiphase flow problem of cells. In all numerical results, our method is verified to be both stable and accurate. In the future work, we will explore the potential of the RK-RFM in other types of PDEs and broader applications.

ACKNOWLEDGMENTS

This research is partially supported by the National Key R&D Program of China (No. 2022YFE03040002) and the National Natural Science Foundation of China (No. 12371434).

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Yangtao Deng: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Investigation (equal); Writing – original draft (equal); Writing – review & editing (equal). **Qiaolin He:** Conceptualization (equal); Investigation (equal); Methodology (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

APPENDIX: FORMULATION OF EQUATION (21)

To better illustrate how to obtain the optimal $\mathbf{U}_{t_{k+1}}$ by the linear least squares method, we consider cases with $d = 2$ and $d_\phi = 1$. The arrangement of subdomains and the distribution of collocation points in subdomain Ω_n are set according to Fig. 2. Now, we have

$$\mathbf{x}^n = \{\mathbf{x}_{1,1}^n, \dots, \mathbf{x}_{1,Q_x}^n, \dots, \mathbf{x}_{Q_y,1}^n, \dots, \mathbf{x}_{Q_y,Q_x}^n\}.$$

The coefficient matrix corresponding to $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ in the subdomain Ω_n is defined as follows:

$$\mathbf{A}_n(\mathbf{x}^n) = \begin{bmatrix} \varphi_{n1,t_{k+1}}(\mathbf{x}_{1,1}^n) & \dots & \varphi_{nJ_n,t_{k+1}}(\mathbf{x}_{1,1}^n) \\ \vdots & \dots & \vdots \\ \varphi_{n1,t_{k+1}}(\mathbf{x}_{Q_y,Q_x}^n) & \dots & \varphi_{nJ_n,t_{k+1}}(\mathbf{x}_{Q_y,Q_x}^n) \end{bmatrix}_{Q \times J_n}.$$

Then, we get the coefficient matrix corresponding to $\tilde{\phi}_{t_{k+1}}(\mathbf{x})$ in the entire domain Ω ,

$$\mathbf{A} = \text{diag}(\mathbf{A}_1(\mathbf{x}^1), \dots, \mathbf{A}_{M_p}(\mathbf{x}^{M_p}))_{M_p Q \times M_p J_n}.$$

Let

$$\begin{aligned} \mathbf{x}_{1,1}^n &= \{\mathbf{x}_{1,1}^n, \dots, \mathbf{x}_{Q_y,1}^n\}, & \mathbf{x}_{1,Q_x}^n &= \{\mathbf{x}_{1,Q_x}^n, \dots, \mathbf{x}_{Q_y,Q_x}^n\}, \\ \mathbf{x}_{1,1}^n &= \{\mathbf{x}_{1,1}^n, \dots, \mathbf{x}_{1,Q_x}^n\}, & \mathbf{x}_{Q_y,1}^n &= \{\mathbf{x}_{Q_y,1}^n, \dots, \mathbf{x}_{Q_y,Q_x}^n\}, \end{aligned}$$

we have the coefficient matrices of the boundary conditions, where the boundary is perpendicular to the x -axis or y -axis in the subdomain Ω_n ,

$$\begin{aligned} \mathbf{B}_{x,n,l} &= [\mathbf{0}_{Q_y \times (n-1)J_n}, \mathbf{B}\mathbf{A}_n(\mathbf{x}_{1,1}^n), \mathbf{0}_{Q_y \times (M_p-n)J_n}]_{Q_y \times M_p J_n}, \\ \mathbf{B}_{x,n,r} &= [\mathbf{0}_{Q_y \times (n-1)J_n}, \mathbf{B}\mathbf{A}_n(\mathbf{x}_{1,Q_x}^n), \mathbf{0}_{Q_y \times (M_p-n)J_n}]_{Q_y \times M_p J_n}, \\ \mathbf{B}_{y,n,d} &= [\mathbf{0}_{Q_x \times (n-1)J_n}, \mathbf{B}\mathbf{A}_n(\mathbf{x}_{1,1}^n), \mathbf{0}_{Q_x \times (M_p-n)J_n}]_{Q_x \times M_p J_n}, \\ \mathbf{B}_{y,n,u} &= [\mathbf{0}_{Q_x \times (n-1)J_n}, \mathbf{B}\mathbf{A}_n(\mathbf{x}_{Q_y,1}^n), \mathbf{0}_{Q_x \times (M_p-n)J_n}]_{Q_x \times M_p J_n}, \end{aligned}$$

where the subscripts l , r , d , and u denote the left boundary, right boundary, lower boundary, and upper boundary, respectively. Therefore, we obtain

$$\begin{aligned} B_x &= \left[B_{x,1,l}^T, \dots, B_{x,N_y,l}^T, B_{x,(N_x-1)N_y+1,r}^T, \dots, B_{x,N_xN_y,r}^T \right]_{2N_yQ_y \times M_pJ_n}^T, \\ B_y &= \left[B_{y,1,d}^T, B_{y,N_y,u}^T, \dots, B_{y,(N_x-1)N_y+1,d}^T, B_{y,N_xN_y,u}^T \right]_{2N_xQ_x \times M_pJ_n}^T, \end{aligned}$$

in the whole domain Ω .

In the subdomain Ω_n , the coefficient matrix of the C^0 continuity condition is as follows, where the interface is perpendicular to the x -axis or y -axis,

$$\begin{aligned} C_{x,n}^0 &= \left[\mathbf{0}_{Q_y \times (n-1)J_n}, \mathbf{A}_n(\mathbf{x}_{Q_x}^n), \mathbf{0}_{Q_y \times N_yJ_n}, \right. \\ &\quad \left. - \mathbf{A}_{n+N_y}(\mathbf{x}_{1,1}^{n+N_y}), \mathbf{0}_{Q_y \times (M_p-n-N_y)J_n} \right]_{Q_y \times M_pJ_n}, \\ C_{y,n}^0 &= \left[\mathbf{0}_{Q_x \times (n-1)J_n}, \mathbf{A}_n(\mathbf{x}_{Q_y}^n), \right. \\ &\quad \left. - \mathbf{A}_{n+1}(\mathbf{x}_{1,1}^{n+1}), \mathbf{0}_{Q_x \times (M_p-n-1)J_n} \right]_{Q_x \times M_pJ_n}. \end{aligned}$$

Then, we obtain that

$$\begin{aligned} C_x^0 &= \left[(C_{x,1}^0)^T, \dots, (C_{x,N_y(N_x-1)}^0)^T \right]_{N_y(N_x-1)Q_y \times M_pJ_n}^T, \\ C_y^0 &= \left[(C_{y,1}^0)^T, \dots, (C_{y,N_y-1}^0)^T, \dots, \right. \\ &\quad \left. (C_{y,N_y(N_x-1)+1}^0)^T, \dots, (C_{y,N_yN_x-1}^0)^T \right]_{N_x(N_y-1)Q_x \times M_pJ_n}^T. \end{aligned}$$

Similarly, we have the following coefficient matrices of the C^1 continuity condition:

$$\begin{aligned} C_x^0, C_x^1 &= \nabla_x C_x^0, \\ C_y^0, C_y^1 &= \nabla_y C_y^0. \end{aligned}$$

The right-hand side term corresponding to $\mathbf{A}_n(\mathbf{x}^n)$ is defined as

$$\mathbf{f}_{A_n}(\mathbf{x}^n) = \left[\phi_{t_{k+1}}(\mathbf{x}_{1,1}^n), \dots, \phi_{t_{k+1}}(\mathbf{x}_{Q_y,Q_x}^n) \right]_{1 \times Q}.$$

Therefore, we have

$$\mathbf{f}_A = \left[\mathbf{f}_{A_1}(\mathbf{x}^1), \dots, \mathbf{f}_{A_{M_p}}(\mathbf{x}^{M_p}) \right]_{M_pQ \times 1}^T,$$

related to A .

Also, we have

$$\begin{aligned} \mathbf{f}_{B_x} &= \left[\mathbf{g}(\mathbf{x}_{1,1}^1, t_{k+1}), \dots, \mathbf{g}(\mathbf{x}_{1,1}^{N_y}, t_{k+1}), \right. \\ &\quad \left. \times \mathbf{g}(\mathbf{x}_{Q_x}^{(N_x-1)N_y+1}, t_{k+1}), \dots, \mathbf{g}(\mathbf{x}_{Q_x}^{N_xN_y}, t_{k+1}) \right]_{N_yQ_y \times 1}^T, \\ \mathbf{f}_{B_y} &= \left[\mathbf{g}(\mathbf{x}_{1,1}^1, t_{k+1}), \mathbf{g}(\mathbf{x}_{Q_y}^{N_y}, t_{k+1}), \dots, \right. \\ &\quad \left. \times \mathbf{g}(\mathbf{x}_{1,1}^{(N_x-1)N_y+1}, t_{k+1}), \mathbf{g}(\mathbf{x}_{Q_y}^{N_xN_y}, t_{k+1}) \right]_{N_xQ_x \times 1}^T, \\ \mathbf{f}_{C_x^0} &= [\mathbf{0}]_{N_y(N_x-1)Q_y \times 1}, \\ \mathbf{f}_{C_y^0} &= [\mathbf{0}]_{N_x(N_y-1)Q_x \times 1}, \\ \mathbf{f}_{C_x^1} &= [\mathbf{0}]_{N_y(N_x-1)Q_y \times 1}, \\ \mathbf{f}_{C_y^1} &= [\mathbf{0}]_{N_x(N_y-1)Q_x \times 1}. \end{aligned}$$

The coefficient matrices and the right-hand side terms can be rewritten by the rescaling parameters,

$$\begin{aligned} \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |A_{i,j}|}, \quad A_{i,j} = \lambda_i A_{i,j}, \quad f_{A,i,1} = \lambda_i f_{A,i,1}, \\ &\quad i = 1, \dots, M_pQ, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |B_{x,i,j}|}, \quad B_{x,i,j} = \lambda_i B_{x,i,j}, \quad f_{B_x,i,1} = \lambda_i f_{B_x,i,1}, \\ &\quad i = 1, \dots, N_yQ_y, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |B_{y,i,j}|}, \quad B_{y,i,j} = \lambda_i B_{y,i,j}, \quad f_{B_y,i,1} = \lambda_i f_{B_y,i,1}, \\ &\quad i = 1, \dots, N_xQ_x, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |C_{x,i,j}^0|}, \quad C_{x,i,j}^0 = \lambda_i C_{x,i,j}^0, \quad f_{C_x^0,i,1} = \lambda_i f_{C_x^0,i,1}, \\ &\quad i = 1, \dots, N_y(N_x-1)Q_y, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |C_{y,i,j}^0|}, \quad C_{y,i,j}^0 = \lambda_i C_{y,i,j}^0, \quad f_{C_y^0,i,1} = \lambda_i f_{C_y^0,i,1}, \\ &\quad i = 1, \dots, N_x(N_y-1)Q_x, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |C_{x,i,j}^1|}, \quad C_{x,i,j}^1 = \lambda_i C_{x,i,j}^1, \quad f_{C_x^1,i,1} = \lambda_i f_{C_x^1,i,1}, \\ &\quad i = 1, \dots, N_y(N_x-1)Q_y, \\ \lambda_i &= \frac{c}{\max_{1 \leq j \leq M_pJ_n} |C_{y,i,j}^1|}, \quad C_{y,i,j}^1 = \lambda_i C_{y,i,j}^1, \quad f_{C_y^1,i,1} = \lambda_i f_{C_y^1,i,1}, \\ &\quad i = 1, \dots, N_x(N_y-1)Q_x. \end{aligned}$$

Then, we attain

$$\mathbf{A}_{t_{k+1}} \mathbf{U}_{t_{k+1}} = \begin{bmatrix} \mathbf{A} \\ \mathbf{B}_x \\ \mathbf{B}_y \\ \mathbf{C}_x^0 \\ \mathbf{C}_y^0 \\ \mathbf{C}_x^1 \\ \mathbf{C}_y^1 \end{bmatrix} \mathbf{U}_{t_{k+1}} = \begin{bmatrix} \mathbf{f}_A \\ \mathbf{f}_{B_x} \\ \mathbf{f}_{B_y} \\ \mathbf{f}_{C_x^0} \\ \mathbf{f}_{C_y^0} \\ \mathbf{f}_{C_x^1} \\ \mathbf{f}_{C_y^1} \end{bmatrix} = \mathbf{f}_{t_{k+1}}, \quad (\text{A1})$$

related to the loss function (20), where $\mathbf{U}_{t_{k+1}} = [u_{11,t_{k+1}}, \dots, u_{1J_n,t_{k+1}}, \dots, u_{M_pJ_n,t_{k+1}}]^T$. Finally, the linear least squares method is employed to solve the overdetermined system of Eq. (A1), and the optimal $\mathbf{U}_{t_{k+1}} = (u_{nj,t_{k+1}})^T$ is obtained.

REFERENCES

- ¹B. Ladoux and R.-M. Mège, "Mechanobiology of collective cell behaviours," *Nat. Rev. Mol. Cell Biol.* **18**, 743–757 (2017).
- ²R. Mueller, J. M. Yeomans, and A. Doostmohammadi, "Emergence of active nematic behavior in monolayers of isotropic cells," *Phys. Rev. Lett.* **122**, 048004 (2019).
- ³R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems* (SIAM, 2007).
- ⁴F. Moukalled, L. Mangani, M. Darwish, F. Moukalled, L. Mangani, and M. Darwish, *The Finite Volume Method* (Springer, 2016).
- ⁵V. Thomée, *Galerkin Finite Element Methods for Parabolic Problems* (Springer Science & Business Media, 2007), Vol. 25.
- ⁶O. C. Zienkiewicz, R. L. Taylor, and J. Zhu, *The Finite Element Method: Its Basis and Fundamentals* (Elsevier, 2005).
- ⁷J. Blazek, *Computational Fluid Dynamics: Principles and Applications* (Butterworth-Heinemann, 2015).

- ⁸Y. Bengio, I. Goodfellow, and A. Courville, *Deep Learning* (MIT Press, 2017).
- ⁹G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Math. Control Signal Syst.* **2**, 303–314 (1989).
- ¹⁰W. E, J. Han, and A. Jentzen, "Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations," *Commun. Math. Stat.* **5**, 349–380 (2017).
- ¹¹J. Han, A. Jentzen, and E. Weinan, "Solving high-dimensional partial differential equations using deep learning," *Proc. Natl. Acad. Sci. U. S. A.* **115**, 8505–8510 (2018).
- ¹²W. E and B. Yu, "The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems," *Commun. Math. Stat.* **6**, 1–12 (2018).
- ¹³J. Sirignano and K. Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," *J. Comput. Phys.* **375**, 1339–1364 (2018).
- ¹⁴Y. Zang, G. Bao, X. Ye, and H. Zhou, "Weak adversarial networks for high-dimensional partial differential equations," *J. Comput. Phys.* **411**, 109409 (2020).
- ¹⁵M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.* **378**, 686–707 (2019).
- ¹⁶W. E, J. Han, and A. Jentzen, "Algorithms for solving high dimensional PDEs: From nonlinear Monte Carlo to machine learning," *Nonlinearity* **35**, 278 (2021).
- ¹⁷W. Zhang, W. Suo, J. Song, and W. Cao, "Physics Informed Neural Networks (PINNs) as intelligent computing technique for solving partial differential equations: Limitation and future prospects," [arXiv:2411.18240](https://arxiv.org/abs/2411.18240) (2024).
- ¹⁸G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, "Extreme learning machine: Theory and applications," *Neurocomputing* **70**, 489–501 (2006).
- ¹⁹R. M. Neal, *Bayesian Learning for Neural Networks* (Springer Science & Business Media, 2012), Vol. 118.
- ²⁰A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems* (Curran Associates Inc., 2007), pp. 1177–1184.
- ²¹G. Huang, G.-B. Huang, S. Song, and K. You, "Trends in extreme learning machines: A review," *Neural Networks* **61**, 32–48 (2015).
- ²²J. Chen, X. Chi, Z. Yang *et al.*, "Bridging traditional and machine learning-based algorithms for solving PDEs: The random feature method," *J. Mach. Learn.* **1**, 268–298 (2022).
- ²³J. Chen and Y. Luo, "The random feature method for time-dependent problems," [arXiv:2304.06913](https://arxiv.org/abs/2304.06913) (2023).
- ²⁴G.-B. Huang, L. Chen, and C.-K. Siew, "Universal approximation using incremental constructive feedforward networks with random hidden nodes," *IEEE Trans. Neural Networks* **17**, 879–892 (2006).
- ²⁵V. Dwivedi and B. Srinivasan, "Physics informed extreme learning machine (PIELM)—A rapid method for the numerical solution of partial differential equations," *Neurocomputing* **391**, 96–118 (2020).
- ²⁶F. Calabrò, G. Fabiani, and C. Siettos, "Extreme learning machine collocation for the numerical solution of elliptic PDEs with sharp gradients," *Comput. Methods Appl. Mech. Eng.* **387**, 114188 (2021).
- ²⁷S. Dong and Z. Li, "Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations," *Comput. Methods Appl. Mech. Eng.* **387**, 114129 (2021).
- ²⁸G. Fabiani, F. Calabrò, L. Russo, and C. Siettos, "Numerical solution and bifurcation analysis of nonlinear partial differential equations with extreme learning machines," *J. Sci. Comput.* **89**, 44 (2021).
- ²⁹Y. Yang, M. Hou, and J. Luo, "A novel improved extreme learning machine algorithm in solving ordinary differential equations by Legendre neural network methods," *Adv. Differ. Equations* **2018**, 469.
- ³⁰J. C. Butcher, "A history of Runge-Kutta methods," *Appl. Numer. Math.* **20**, 247–260 (1996).